

ECOM EXPERTS

EcomWP Merchant Center Software

Manual de usuario e instalación

ecomwpmerchant 1.1.0 (plugin WordPress/WooCommerce) · 06/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

4. Configuración

Google Product Category

Seguridad: Token Cron

Modo Depuración (Logs)

Los dos recuadros de URL

5. Cómo se usa, día a día

Caso A: feed en tiempo real (el más sencillo)

Caso B: fichero en disco regenerado por cron

Qué producto entra y cuál no

Qué lleva cada <item>;

6. Qué ve el cliente en la tienda

7. Preguntas frecuentes y problemas

8. Seguridad

Qué se ha revisado

Qué se ha arreglado en esta revisión

Qué queda pendiente (no se ha tocado porque cambia el comportamiento)

9. Ficha técnica

Plugin de **WordPress + WooCommerce** (no es un módulo de PrestaShop) que expone el catálogo de la tienda como un feed XML para **Google Merchant Center**, con todos los productos clasificados en la categoría de Google «Software».

Versión del plugin: 1.1.0 · Carpeta: `ecomwpmerchant` · Autor: Ecom Experts.

1. Qué hace y para quién

Google Merchant Center necesita recibir los productos de la tienda en un formato concreto (un XML tipo RSS 2.0 con los atributos `g:`). Este plugin lo genera a partir de los productos publicados de WooCommerce, sin exportaciones manuales ni hojas de cálculo: se copia una URL en Merchant Center y Google la lee solo, tantas veces al día como se le indique.

Está pensado para tiendas que venden **software, licencias o productos digitales** y quieren que todos sus artículos lleven la misma categoría de Google («Software > Software de computadora», configurable). Por eso no intenta deducir la categoría de Google por producto: la aplica a todos. Además marca cada producto como nuevo (`new`), en stock (`in_stock`), usa el SKU como MPN y el nombre de la tienda como marca, y deja fuera automáticamente los productos sin precio o sin imagen destacada (Merchant Center los rechazaría igualmente).

Ofrece dos formas de servir el feed: **en tiempo real** (la URL genera el XML en cada petición, siempre al día) y **en disco** (una URL con token escribe `feed.xml` en la carpeta de subidas, para que Google descargue un fichero estático ligero). Todo lo que se omite queda apuntado en un registro de depuración que se puede activar y consultar desde la misma pantalla.

¿Sigue vigente o está superado? Está en `old modules` y hay que decirlo con claridad:

- El **formato** que produce (RSS 2.0 con el espacio de nombres `g:` de Google, recuperación programada por URL) **sigue siendo aceptado por Merchant Center**, y el plugin funciona en WordPress 6.x, WooCommerce 8/9 con HPOS y PHP 7.4-8.3 (comprobado en esta revisión con `php -1` y una batería de pruebas con la API de WordPress simulada).
- Para una tienda «normal» está **superado** por el plugin oficial *Google for WooCommerce* (antes *Google Listings & Ads*), que sincroniza por API sin feed, mapea categorías producto a producto y gestiona GTIN, variaciones, envío e impuestos.
- Sigue teniendo sentido** en su nicho: tiendas de software/licencias que quieren forzar una única categoría de Google, no quieren instalar un plugin pesado con cuenta de Google Ads vinculada, o necesitan un feed por URL para importarlo en otros sitios (Bing Merchant, comparadores). Es decir: vigente como herramienta sencilla de feed, no como integración completa con Merchant Center.

2. Compatibilidad

Requisito	Valor
WordPress	5.5 o superior (<code>esc_xml()</code> existe desde 5.5; si faltara, se usa un escape propio)
WooCommerce	3.0 o superior. Compatible con HPOS (almacenamiento de pedidos en tablas propias): el plugin lo declara y no toca pedidos
PHP	7.4, 8.0, 8.1, 8.2 y 8.3 (comprobado con <code>php -1</code> en 7.4 y 8.1, y ejecutado en 7.4 y 8.3)
Multisitio	Funciona por sitio: las opciones son de cada blog (<code>get_option</code>) y el feed sale de la carpeta de subidas de ese sitio. No hay pantalla de red
Idiomas	Interfaz y textos del feed en castellano, fijos (no lleva ficheros de traducción)
Dependencias	Ninguna externa: sin Composer, sin librerías. Declara <code>Requires Plugins: woocommerce</code> para que WordPress 6.5+ no permita activarlo sin WooCommerce
Servidor web	El <code>.htaccess</code> que protege la carpeta <code>logs/</code> solo actúa en Apache/LiteSpeed. En Nginx hay que denegar esa ruta en la configuración del servidor (ver sección 8)

3. Instalación

1. Subir la carpeta `ecomwpmerchant` a `wp-content/plugins/` (o el ZIP desde *Plugins* → *Añadir nuevo* → *Subir plugin*).
2. Activar **EcomWP Merchant Center Software** en *Plugins* → *Plugins instalados*. Si WooCommerce no está activo, aparece un aviso rojo en el escritorio y el feed responde con un error 503 en texto plano en vez de un XML vacío.
3. Ir a **WooCommerce** → **Merchant XML Feed**.

Qué crea al activarse (y cada vez que se entra en el escritorio, para reparar lo que falte):

- Tres opciones en `wp_options`: `ecomwp_merchant_category` (categoría de Google), `ecomwp_merchant_token` (token aleatorio de 32 caracteres hexadecimales para la URL de generación en disco) y `ecomwp_merchant_debug_mode` (0/1).
- La carpeta `wp-content/uploads/ecomwp-merchant/` con un `index.php` de redirección. Aquí se escribe `feed.xml`. **No** lleva `.htaccess`, porque Google tiene que poder descargarlo.
- La carpeta `wp-content/plugins/ecomwpmerchant/logs/` con `index.php` y un `.htaccess` que deniega todo acceso por URL a los registros.

No crea tablas, ni tareas de WP-Cron, ni páginas, ni reglas de reescritura: las dos URL del feed son parámetros de consulta sobre la portada (`?ecomwp_merchant_xml=1`), así que no hace falta regenerar los enlaces permanentes.

Al desactivar no se borra nada: el feed sigue en disco y la configuración se conserva. **Al desinstalar** (*Eliminar* desde el listado de plugins) `uninstall.php` borra las tres opciones, la carpeta `uploads/ecomwp-merchant/` con el `feed.xml` y la carpeta `logs/`. Si Google todavía tiene configurada la URL del fichero, empezará a recibir un 404: hay que quitar el feed en Merchant Center antes o después.

4. Configuración

Todo está en una sola pantalla: **WooCommerce** → **Merchant XML Feed**. Solo la ven y la guardan los usuarios con la capacidad `manage_woocommerce` (administradores y gestores de tienda).

No hay capturas de esta pantalla: este proyecto no tiene una instalación de WordPress local donde generarlas. La pantalla se describe a partir del código.

Google Product Category

Cadena que se escribe tal cual en `<g:google_product_category>` de **todos** los productos. Por defecto `Software > Software de computadora`. Puede ponerse el nombre completo de la categoría o su **número** de la taxonomía de Google; el número (por ejemplo `313`, que es `Software > Computer Software`) es lo más seguro, porque no depende del idioma del feed ni de cómo esté escrito el nombre. Se guarda como texto plano: cualquier etiqueta HTML se elimina y, si se deja vacío, vuelve al valor por defecto.

Seguridad: Token Cron

Clave que protege la URL de generación en disco. Se genera sola al activar. Se puede cambiar, pero debe tener **al menos 16 caracteres, solo letras y números**; si se pone algo más corto o con símbolos, el plugin genera uno nuevo y lo avisa en amarillo al guardar (antes, un token vacío dejaba la URL de generación abierta a cualquiera). Al cambiarlo hay que actualizar la URL en el cron del servidor.

Modo Depuración (Logs)

Casilla. Activa la escritura de `logs/merchant_log_AAAA-MM-DD.log` con los productos que se omiten del feed y el motivo (precio 0, sin imagen destacada, imagen sin URL), los intentos de acceso al cron con token incorrecto y el resultado de cada escritura de `feed.xml`. Con la casilla apagada no se escribe nada. Las últimas 20 líneas del registro de hoy se ven al pie de la misma pantalla, en el recuadro negro «Últimos Logs de Depuración».

Los dos recuadros de URL

Debajo del formulario aparecen las dos direcciones ya montadas para copiar:

1. **Feed en Tiempo Real:** `https://su-tienda.com/?ecomwp_merchant_xml=1` , con el botón *Ver XML vivo* para abrirlo en otra pestaña.
2. **URL de Cron (Generación en Disco):** `https://su-tienda.com/?ecomwp_merchant_cron=1&token=...` , y debajo el enlace al fichero resultante `https://su-tienda.com/wp-content/uploads/ecomwp-merchant/feed.xml` .

5. Cómo se usa, día a día

Caso A: feed en tiempo real (el más sencillo)

1. Abrir *Ver XML vivo* y comprobar que salen los productos esperados. Si falta alguno, activar el modo depuración, recargar el feed y mirar el registro: dirá por qué se omitió.
2. En Google Merchant Center, *Productos* → *Feeds* → *Añadir feed*, tipo **Recuperación programada**, pegar la URL del recuadro 1 y elegir la frecuencia (diaria suele bastar).
3. Cada vez que Google descarga, el plugin consulta WooCommerce en ese momento: precios y stock siempre actuales, sin nada que programar en el servidor.

Contra: cada descarga recorre el catálogo entero (`limit => -1`). En tiendas con miles de productos puede tardar varios segundos y consumir memoria; para esas, el caso B.

Caso B: fichero en disco regenerado por cron

1. Programar en el servidor (cPanel, Plesk, `crontab`) una llamada periódica a la URL del recuadro 2, por ejemplo cada noche: `curl -s "https://su-tienda.com/?ecomwp_merchant_cron=1&token=XXXX" > /dev/null` La respuesta es texto plano: `XML generado con éxito.` (200), `Acceso denegado: token inválido.` (403) o un 500 si la carpeta de subidas no es escribible.
2. Dar a Google la URL del fichero `.../uploads/ecomwp-merchant/feed.xml` como recuperación programada, a una hora posterior a la del cron.
3. El fichero se escribe primero en `feed.xml.tmp` y después se renombra, así Google nunca descarga un XML a medias aunque coincida con la regeneración.

Qué producto entra y cuál no

Entran los productos **publicados**, con **visibilidad en catálogo** y **en stock** en WooCommerce. De esos se descartan, dejándolo en el registro si la depuración está activa:

- Precio 0 o negativo (`get_price()`): Google no admite productos gratuitos en Shopping.
- Sin imagen destacada, o con una imagen cuyo adjunto ya no existe.

Los productos variables salen **una vez, como el producto padre**, con el precio que WooCommerce devuelve para el padre (el mínimo del rango) y la imagen destacada del padre. No se generan `item` por variación ni `g:item_group_id` .

Qué lleva cada <item>

Etiqueta	De dónde sale
<code>g:id</code>	ID del producto en WooCommerce
<code>g:title</code>	Nombre del producto
<code>g:description</code>	Descripción corta; si está vacía, la larga. Sin HTML ni shortcodes
<code>g:link</code>	Enlace permanente del producto
<code>g:image_link</code>	Imagen destacada a tamaño completo
<code>g:condition</code>	Siempre <code>new</code>
<code>g:availability</code>	Siempre <code>in_stock</code> (solo entran productos en stock)
<code>g:price</code>	Precio con dos decimales y la moneda de WooCommerce (<code>49.90 EUR</code>)
<code>g:google_product_category</code>	La categoría configurada, igual para todos
<code>g:product_type</code>	Las categorías de WooCommerce del producto, unidas con <code>></code> (si tiene)
<code>g:brand</code>	El nombre del sitio (<i>Ajustes → Generales → Título del sitio</i>)
<code>g:mpn</code> + <code>g:identifier_exists=yes</code>	El SKU, si el producto lo tiene
<code>g:identifier_exists=no</code>	Si no tiene SKU

El precio es el que devuelve `get_price()` : si la tienda muestra precios con impuestos incluidos en el catálogo pero los guarda sin ellos, Merchant Center puede marcar discrepancias de precio. Es una limitación conocida (ver sección 7).

6. Qué ve el cliente en la tienda

Nada. El plugin no añade nada al front de la tienda: solo responde a las dos URL con parámetro (`?ecomwp_merchant_xml=1` y `?ecomwp_merchant_cron=1`) y, en el resto de peticiones, no hace nada. Ambas respuestas llevan cabeceras anti-caché (`nocache_headers()`) para que un plugin de caché o un CDN no sirva un feed viejo.

7. Preguntas frecuentes y problemas

Abro la URL del feed y sale la portada de la tienda, no un XML. Otro plugin (caché de página, seguridad) está sirviendo la portada cacheada antes de que WordPress ejecute `init` . Excluir en ese plugin las URL con `ecomwp_merchant_xml` y `ecomwp_merchant_cron` , o usar el caso B (archivo estático).

Sale «WooCommerce no está activo: no se puede generar el feed» (503). WooCommerce está desactivado o roto. Activarlo.

Falta un producto en el feed. Activar *Modo Depuración* , recargar el feed y leer el recuadro de logs: indicará `ID N omitido:` `Precio 0...` o `...No tiene imagen destacada` . Si no aparece, el producto no está publicado, no es visible en el catálogo o está sin stock.

Merchant Center dice «valor no válido» en `google_product_category` . Escribir el **número** de la categoría (por ejemplo `313`) en lugar del nombre. Los nombres dependen del idioma de la taxonomía y de los espacios alrededor de `>` .

Merchant Center avisa de falta de GTIN. El plugin declara `identifier_exists=yes` con marca + MPN (SKU) cuando hay SKU, y `no` cuando no lo hay. Para software sin código de barras esto es lo aceptado por Google; si Google exige GTIN para alguna referencia concreta, no hay campo para ello en esta versión.

El cron responde 403. El token de la URL no coincide con el guardado (se regeneró al guardar con un valor inválido, o se cambió). Copiar la URL nueva del recuadro 2.

El cron responde 500. La carpeta `wp-content/uploads/ecomwp-merchant/` no es escribible por PHP. Revisar permisos de `uploads`.

Los precios del feed no cuadran con los de la tienda. El feed usa el precio base que devuelve WooCommerce, sin aplicar la visualización con/sin impuestos. Si la tienda muestra impuestos incluidos y los guarda sin incluir, habrá diferencia; en esta versión no hay opción para elegirlo.

¿Se puede tener más de una categoría de Google? No. Este plugin aplica una sola a todo el catálogo; ese es su propósito. Para categorías por producto, usar *Google for WooCommerce*.

8. Seguridad

Qué se ha revisado

Los siete ficheros PHP del plugin, línea a línea: `ecomwpmerchant.php`, `includes/class-ecomwpmerchant.php`, `includes/class-ecomwpmerchant-admin.php`, `includes/class-ecomwpmerchant-feed.php`, `includes/class-ecomwpmerchant-logger.php`, `uninstall.php` y los `index.php`. No hay JavaScript, CSS ni plantillas aparte.

Área	Estado
Nonce y capacidades en la administración	El formulario va por <code>options.php</code> con <code>settings_fields()</code> (nonce de WordPress) y los tres ajustes se registran con <code>register_setting()</code> y su <code>sanitize_callback</code> . El menú y la pantalla exigen <code>manage_woocommerce</code> . No hay endpoints AJAX ni REST
Escapado de salida	Todo lo que se imprime en la pantalla pasa por <code>esc_attr</code> , <code>esc_url</code> o <code>esc_html</code> ; el XML sale por <code>esc_xml()</code> (o <code>htmlspecialchars</code> con <code>ENT_XML1</code> si no existe) y <code>esc_url</code>
Saneado de entrada	<code>sanitize_text_field(wp_unslash())</code> en los parámetros GET; saneadores propios en las tres opciones
SQL	El plugin no ejecuta ninguna consulta propia: usa <code>wc_get_products()</code> , <code>get_term()</code> y la API de opciones
Subida de ficheros	No hay subida. La única escritura es <code>feed.xml</code> , con nombre fijo, en una carpeta fija, atómica (temporal + <code>rename</code>)
Recorrido de rutas	No se construye ninguna ruta con datos del usuario
Ejecución de código	No hay <code>eval</code> , <code>exec</code> , <code>unserialize</code> ni similares
Secretos	El único secreto es el token del cron, generado con <code>random_bytes()</code> , guardado en <code>wp_options</code> y comparado con <code>hash_equals()</code> . No hay claves en el código
Datos enviados a terceros	El plugin no envía nada a nadie. Es Google quien descarga el feed; lo que expone son datos ya públicos de productos publicados y visibles (nombre, descripción, precio, enlace, imagen, SKU). No sale ningún dato de clientes ni de pedidos, y no se usa ninguna credencial de Google
Fugas	Sin <code>var_dump</code> / <code>print_r</code> . <code>logs/</code> con <code>index.php</code> y <code>.htaccess</code> de denegación total. La carpeta del XML lleva <code>index.php</code> (sin <code>.htaccess</code> , porque el feed debe servirse)
Anti-caché	<code>nocache_headers()</code> en las dos URL
PHP 8	<code>wc_get_products()</code> , <code>get_category_ids()</code> , <code>file()</code> y <code>glob()</code> comprobados con <code>is_array()</code> antes de iterar

Qué se ha arreglado en esta revisión

Trabajo en dos sesiones (la primera se cortó por cuota y dejó los cambios sin probar; esta los ha completado y verificado).

Fichero	Arreglo	Por qué
<code>includes/class-ecomwpmerchant-feed.php</code>	Token del cron comparado con <code>hash_equals()</code> , rechazado si está vacío o tiene menos de 16 caracteres	Antes, un token vacío en la base de datos y <code>&token=</code> vacío en la URL pasaban la comprobación y cualquiera podía forzar la escritura del feed
<code>includes/class-ecomwpmerchant-feed.php</code>	Parámetros GET saneados con <code>sanitize_text_field(wp_unslash())</code> ; respuesta 503 explícita si WooCommerce no está activo; cabeceras <code>nocache_headers()</code> ; escritura atómica de <code>feed.xml</code> con <code>temporal + rename</code> y comprobación del resultado (500 si falla)	Antes el XML podía quedar a medias en disco, no había anti-caché y sin WooCommerce daba un fatal
<code>includes/class-ecomwpmerchant-feed.php</code>	Escapado de todos los textos del XML con <code>esc_xml()</code> (con alternativa <code>ENT_XML1</code>), incluido el nombre de la tienda en el título del canal; descripciones sin shortcodes (<code>strip_shortcodes</code>) ni HTML; se omiten productos cuya imagen destacada no tiene URL	Un <code>&</code> o un <code><</code> en un nombre de producto rompía el XML entero; los shortcodes de maquetadores salían como texto en Google
<code>includes/class-ecomwpmerchant-feed.php</code>	El <code>exit</code> de fin de petición se ha aislado en <code>terminar()</code>	Permite probar el feed sin WordPress (véase <code>utiles/ecomwpmerchant/prueba_stubs.php</code>); en producción hace exactamente lo mismo
<code>includes/class-ecomwpmerchant-admin.php</code>	<code>sanitize_callback</code> en los tres ajustes (categoría en texto plano y nunca vacía; token alfanumérico de 16+ caracteres o regenerado con aviso; casilla 0/1). <code>current_user_can('manage_woocommerce')</code> también dentro de la pantalla. <code>settings_errors()</code> para mostrar el aviso del token. <code>ecomwpmerchant_tail()</code> comprueba <code>is_array()</code> sobre <code>file()</code>	Sin saneadores, cualquier cadena entraba tal cual en la base de datos y en el XML
<code>includes/class-ecomwpmerchant.php</code> + <code>-admin.php</code>	Filtro <code>option_page_capability_ecomwp_merchant_settings</code> → <code>manage_woocommerce</code>	<code>options.php</code> exige <code>manage_options</code> por defecto: un gestor de tienda veía la pantalla pero al guardar recibía «No tienes permiso»; ahora la misma capacidad abre y guarda
<code>ecomwpmerchant.php</code>	Función <code>ecomwpmerchant_proteger_carpeta()</code> : <code>index.php</code> en la carpeta del XML; <code>index.php + .htaccess</code> de denegación en <code>logs/</code> . Aviso en el escritorio si WooCommerce no está activo. Declaración de compatibilidad con HPOS. Cabecera <code>Requires Plugins: woocommerce</code>	Los <code>.log</code> eran legibles por URL conociendo la fecha; WooCommerce marcaba el plugin como incompatible con HPOS
<code>includes/class-ecomwpmerchant-logger.php</code>	Reutiliza la protección de carpeta al crear <code>logs/</code>	Si el primer log se escribía antes de que existiera la carpeta, quedaba sin <code>.htaccess</code>
<code>uninstall.php</code> (nuevo)	Borra las tres opciones, <code>uploads/ecomwp-merchant/</code> y <code>logs/</code>	Antes desinstalar dejaba opciones, token y feed en el servidor
<code>logs/index.php</code> (nuevo, en el paquete)	Fichero de redirección con cabecera de licencia	Que la carpeta exista ya protegida desde el ZIP, sin depender de la activación

Todo verificado con `php -l` en PHP 7.4 y 8.1 (sin errores en los ocho ficheros) y con `utiles/ecomwpmerchant/prueba_stubs.php`, una prueba con la API de WordPress/WooCommerce simulada que ejecuta el

plugin de verdad y comprueba 33 puntos (hooks registrados, token, carpetas protegidas, saneadores, XML válido y escapado, omisión de productos, cron con token malo/vacío/bueno, escritura atómica, peticiones ajenas intactas y `uninstall.php`). Resultado: **FALLOS: 0** tanto en PHP 8.3 como en PHP 7.4.

Qué queda pendiente (no se ha tocado porque cambia el comportamiento)

- **Sin límite de peticiones en el feed vivo.** `/?ecomwp_merchant_xml=1` es pública (tiene que serlo para Google) y recorre el catálogo entero en cada llamada. Alguien podría martillearla para cargar el servidor. Mitigación hoy: usar el caso B (fichero estático) o limitar la URL en el cortafuegos/CDN. Una versión futura podría cachear el XML en un transitorio durante unos minutos.
- **Token en la URL del cron.** Queda en los registros de acceso del servidor y del proxy. Es lo habitual en este tipo de URL; si preocupa, regenerar el token periódicamente.
- **logs/ solo está protegida en Apache/LiteSpeed.** En Nginx añadir `location ~* /wp-content/plugins/ecomwpmerchant/logs/ { deny all; }`.
- **Los logs no rotan.** Un fichero por día, sin borrado automático; con la depuración apagada (el estado por defecto) no crece nada.
- **El feed lleva el precio base,** sin la lógica de impuestos de visualización de WooCommerce (ver sección 7).

9. Ficha técnica

Ficheros

```
ecomwpmerchant/
ecomwpmerchant.php      cabecera del plugin, protección de carpetas, arranque
uninstall.php           limpieza al desinstalar
index.php
includes/
class-ecomwpmerchant.php EcomWP_Merchant: registra hooks
class-ecomwpmerchant-admin.php EcomWP_Merchant_Admin: menú, ajustes, pantalla
class-ecomwpmerchant-feed.php EcomWP_Merchant_Feed: URL, XML, escritura en disco
class-ecomwpmerchant-logger.php EcomWP_Merchant_Logger: registro en logs/
index.php
logs/                   registros (index.php + .htaccess)
```

Opciones (wp_options): `ecomwp_merchant_category` (texto), `ecomwp_merchant_token` (texto alfanumérico ≥ 16), `ecomwp_merchant_debug_mode` (0/1). Grupo de ajustes `ecomwp_merchant_settings`.

Hooks que usa: `admin_menu` (submenú bajo `woocommerce`, slug `ecomwp-merchant-feed`, capacidad `manage_woocommerce`), `admin_init` (`register_setting`), filtro `option_page_capability_ecomwp_merchant_settings`, `init` (captura de las dos URL), `admin_notices` (aviso sin WooCommerce), `before_woocommerce_init` (HPOS), `register_activation_hook`.

URL

URL	Método	Acceso	Respuesta
<code>/?ecomwp_merchant_xml=1</code>	GET	pública	<code>application/xml</code> , feed completo
<code>/?ecomwp_merchant_cron=1&token=...</code>	GET	token	<code>text/plain</code> : 200 «XML generado con éxito.», 403 token inválido, 500 no se pudo escribir, 503 sin WooCommerce
<code>/wp-content/uploads/ecomwp-merchant/feed.xml</code>	GET	pública	fichero estático generado por la anterior

Tablas: ninguna propia. **WP-Cron:** ninguno (el cron es externo, por URL). **Reglas de reescritura:** ninguna.

Registro: `wp-content/plugins/ecomwpmerchant/logs/merchant_log_AAAA-MM-DD.log`, solo con `ecomwp_merchant_debug_mode = 1`. Formato `[fecha hora] [NIVEL] mensaje`. Niveles usados: `INFO`, `WARNING`, `ERROR`.

Permisos de escritura necesarios: `wp-content/uploads/ecomwp-merchant/` (feed) y `wp-content/plugins/ecomwpmerchant/logs/` (registros).

Pruebas: `utils/ecomwpmerchant/prueba_stubs.php` (fuera del paquete del plugin): `php prueba_stubs.php` → debe terminar en `FALLOS: 0`.